ROS2 Spring Workshop

Lab 2: Writing Your First Robot Brain

Project Context: Your First Autonomous Node

In Lab 1, you drove the turtle by pressing arrow keys. That worked because a *human* was in the loop making decisions 30 times a second. In the Final Project, the tractor will drive itself for minutes at a time with no human input; it needs its own brain.
Today you write the skeleton of every autonomous node you will build this semester: a Python class that wakes up periodically, thinks, and publishes commands. Every lab from here on layers more intelligence onto this pattern.

Objective

In this lab, you will write a **Python node** to drive the turtle automatically.

You will learn:

- **Publishers:** How to send commands to a robot.
- **Timers:** How to make a robot perform actions repeatedly.
- **Package creation:** How to organize your code files.
- **Safe shutdown:** Why every controller must stop cleanly on Ctrl+C.

Pre-Lab Concept Primer: The Anatomy of a Node

Before writing code, we need to understand the structure. Every autonomous node you build this semester will follow the same pattern, so it's worth slowing down here.

1. Why OOP for Robots?

A robot is a thing that exists over time. It has **state** (where am I? what step am I on?), it has **behaviours** (drive, turn, stop), and it has to **persist**. It can't forget itself between actions. Plain functions don't capture this; they run once, lose everything, and exit.

Object-Oriented Programming (OOP) bundles state and behaviour together into a single long-lived "thing." In our case, that thing is the robot.

2. Class vs. Object

- **The Class (blueprint):** The code in your `.py` file. It describes *how* a robot of this type behaves: what it can do, what state it tracks. By itself, the class is just text on disk. Nothing is running yet.
- **The Object (the running robot):** When the program starts, `main()` "instantiates" the class. It builds a live instance from the blueprint. This object is the thing actually executing methods and holding state. One blueprint can build many robots.

3. What is `self`?

`self` means **"this specific robot."** Whenever you see `self.something`, read it as "this robot's something."

- `self.step = 0`, this robot's step counter is now 0.

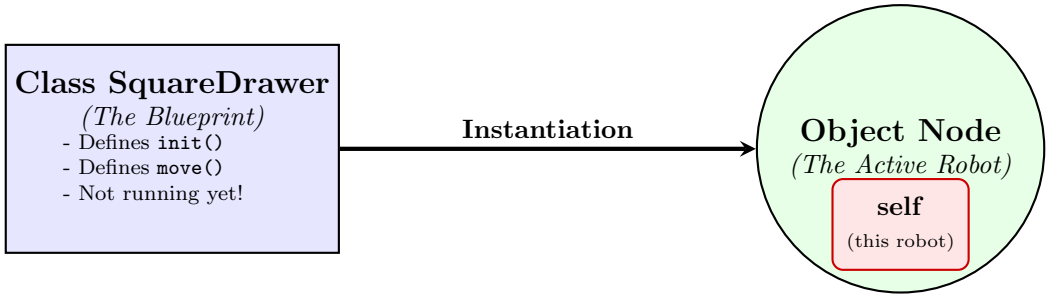

Figure 1: The class is a plan. The object is the actual running robot. `self` is how the robot refers to itself from inside its own code.

- `self.publisher_` this robot's publisher.
- `self.move_turtle_logic()` this robot, running its move logic.

The contrast with plain variables is what makes this powerful:

- `x = 5` inside a function is local. When the function returns, `x` disappears (amnesia).
- `self.x = 5` is attached to the robot. It survives between method calls, and the robot remembers it.

This is how the robot keeps its identity across time. The timer fires, a method runs, it updates `self.step`, then exits. Next time the timer fires, `self.step` is still there, because it lives on the robot, not in the function.

4. The ROS 2 Toolkit (Pubs, Subs, and Timers)

A node by itself is just an object sitting in memory. To do anything useful in ROS 2, it needs three tools:

- **Publisher (the mouth):** Shouts data (e.g. "velocity = 2.0") onto a topic.
- **Subscriber (the ear):** Listens to a topic. When data arrives, it triggers a function.
- **Timer (the heartbeat):** An alarm clock that wakes the robot up periodically to think.

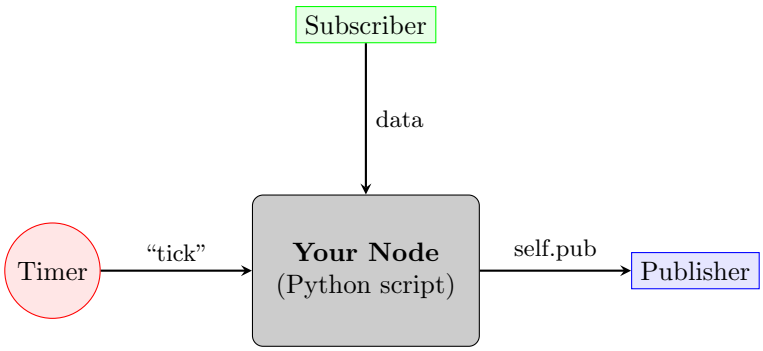

Figure 2: Architecture: the timer wakes the node, the node decides what to do, the publisher sends the command out.

In today's lab you'll wire up a timer and a publisher. Subscribers come in Lab 3.

5. The Syntax Breakdown (How to Speak ROS)

You'll see these three calls constantly. Here's exactly what each argument means.

1. The Publisher (Speaking)

`self.create_publisher(MsgType, 'TopicName', QueueSize)`

- **MsgType (Twist):** The message format. `Twist` carries velocity (linear X, angular Z).
- **'TopicName' (/cmd_vel):** The channel we're shouting on.
- **QueueSize (10):** If we publish faster than anyone reads, keep the last 10 messages buffered.

2. The Timer (Waking Up)

`self.create_timer(Interval, CallbackFunction)`

- **Interval (1.0):** Seconds between ticks (1.0 s = 1 Hz).
- **CallbackFunction (self.move_logic):** The function to run on each tick. *Note: no parentheses. You're handing ROS the function itself, not its result.*

3. The Logger (Status Updates)

`self.get_logger().info/warn/error("Message")`

- **info:** General status ("Node has started"). White text.
- **warn:** Yellow alerts when something is off but not fatal ("Geofence breached").
- **error:** Red alerts for critical failures.

Part 1: Workspace Setup

We need to create a structured home for your code. In ROS 2, we don't just make loose files; we make **packages**.

Create the Package

Now that the workspace is clean, generate the new package structure:

```
# Still inside the src folder...
ros2 pkg create --build-type ament_python lab2_turtlesim_chase
```

Why do we do this?
The flag `--build-type ament_python` tells ROS that we are writing Python scripts, not C++. This sets up the folder structure automatically so we don't have to do it manually.

Part 2: The "Square Drawer" Script

Step A: Create the File

In ROS 2, Python scripts must live inside the inner "module" folder. This is the second folder with the same name as your package.

```
# Navigate into the inner python source folder
cd lab2_turtlesim_chase/lab2_turtlesim_chase

# Create and open the python file
code draw_square.py
```

Step B: Write the Code

This code is in the lab resources folder. You can use the terminal (use command `cp [source] [destination]`) or the GUI to copy it to your new file `draw_square.py`.

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class SquareDrawer(Node):
    def __init__(self):
        # Give the node a name for the logs
        super().__init__('square_drawer_node')

        # 1. PUBLISHER: Speak "Twist" messages to '/turtle1/cmd_vel'
        self.publisher_ = self.create_publisher(
            Twist, '/turtle1/cmd_vel', 10)

        # 2. TIMER: Wake up 'move_turtle_logic' every 1.0 seconds
        self.timer = self.create_timer(1.0, self.move_turtle_logic)

        self.step = 0
        self.get_logger().info("Square Drawer Node has started!")

    def move_turtle_logic(self):
        # Create a new message to send
        msg = Twist()

        # Logic: Even steps = Drive, Odd steps = Turn
        # This uses the Modulo operator (%) to check for remainders
        if self.step % 2 == 0:
            msg.linear.x = 2.0    # Drive forward (m/s)
            msg.angular.z = 0.0
            self.get_logger().info("Driving forward...")
        else:
            msg.linear.x = 0.0
            msg.angular.z = 1.57  # Turn 90 degrees (approx) in radians
            self.get_logger().info("Turning...")

        # Send the message using the publisher we saved in 'self'
        self.publisher_.publish(msg)
        self.step += 1


def main(args=None):
    rclpy.init(args=args)
    node = SquareDrawer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        # SAFETY: publish zero velocity on shutdown so the turtle
        # doesn't coast after Ctrl+C. You will use this pattern in
        # every robot controller you write this semester.
        stop_msg = Twist()
        node.publisher_.publish(stop_msg)
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Listing 1: draw_square.py

Code Walkthrough: Chunk by Chunk

Let's break down exactly what this script is doing:

- **Lines 1-4 (The Shebang & Setup):** Line 1 contains the "shebang" (`#!/usr/bin/env python3`). This is a special instruction that tells Linux exactly which interpreter to use to run the file. Without it, the operating system wouldn't know it's a Python script when ROS tries to run it, because Linux doesn't use file extensions. Lines 2-4 import our tools: the ROS 2 Python client library (`rclpy`), the base `Node` class, and the `Twist` message type (which handles standard velocity commands).

- **Lines 6-19 (The Constructor):** The `__init__` method runs exactly once when the node is created. It names the node `'square_drawer_node'`, creates our Publisher (the "mouth"), and sets up a Timer (the "heartbeat") to trigger the `move_turtle_logic` function exactly once every 1.0 seconds. We also initialize a counter (`self.step = 0`) to keep track of what action we are on.
- **Lines 21-38 (The Brains):** The `move_turtle_logic` function is our callback. It runs every time the 1.0-second timer ticks. It uses the modulo operator (`% 2`) to alternate actions: on even steps it drives straight (`linear.x = 2.0`), and on odd steps it spins (`angular.z = 1.57`). Finally, it publishes this command to the robot and increments the step counter so the next tick does the opposite action.
- **Lines 41-59 (The Engine & Brakes):** The `main()` function acts as the ignition. The command `rclpy.spin(node)` keeps the program alive, allowing the timer to continuously tick. Crucially, if you press Ctrl+C (`KeyboardInterrupt`), the code jumps down to the `finally` block. It creates a blank, zero-velocity `Twist` message and publishes it to force the robot to stop before the node is destroyed.

Professional Habit: Always Stop On Shutdown

Notice the `try/except/finally` block in `main()`. When you Ctrl+C, the `finally` block publishes a zero-velocity `Twist` before the node exits.
Why it matters: Without this, the turtle (or a real robot in later labs) would keep driving with its last command until something else stops it. On the physical tractor in Lab 8, this is a genuine safety issue.
Get in the habit now, because you will reuse this pattern in Labs 3, 6, 7, 8, 9, and 10.

AgTech Alert: Tank vs. Tractor Physics

Critical concept for Lab 8:
In this code, you told the turtle to spin in place:

- `msg.linear.x = 0.0` (stop)
- `msg.angular.z = 1.57` (spin)

Why this is dangerous later:
Turtlesim acts like a tank (differential drive). It can spin on a dime. Your physical AgBot (Lab 8) acts like a tractor (Ackermann steering). It **cannot** spin in place. If you try to turn the wheels without moving forward on the real robot, you will burn out the steering servo. On a tractor, a turn must always be an arc (`linear.x` > 0).
Today we're in simulation, so the spin-in-place works fine. But remember this when you move to real hardware.

Part 3: Registration & Build

Creating a file isn't enough. We must "register" the script so ROS 2 can find it, and then "build" it to install it into the system.

Step A: Modify setup.py

1. Open `src/lab2_turtlesim_chase/setup.py`.
2. Find the `entry_points` section. This works like a **phonebook**. We are telling ROS: *"When I type the command `draw_square`, go look inside the file `draw_square.py` and run the `main` function."*
3. Update it to look like this:

```
entry_points={
    'console_scripts': [
        'draw_square = lab2_turtlesim_chase.draw_square:main',
    ],
},

```

Step B: Build the Workspace

We must compile the workspace to move our script from the "source" folder to the "install" folder where ROS can see it.

1. Go back to the root folder and run `colcon build`:

```
cd /workspaces/agtech_ros2
colcon build

```

2. **Source the setup file:** This refreshes your terminal so it sees the new program.

```
source install/setup.bash

```

Stop! Read This Before Building

By default, ROS 2 creates a *copy* of your python script when you build. If you edit the file in VS Code, ROS 2 keeps running the old copy, and you will think you are going crazy because your changes won't show up.
The fix: We use a special flag to create a "live link" (symlink) between your code and ROS.
Run this command (and memorize it):

```
cd /workspaces/agtech_ros2
colcon build --symlink-install
source install/setup.bash
```

What does this do?

- **Standard build:** Copy/paste file (static). Requires rebuild on every edit.
- **Symlink build:** Creates a shortcut (dynamic). You can edit Python code and run it immediately without rebuilding!

Part 4: The Test Drive

Now we run the system. We need two terminals: one for the robot (simulator) and one for the brain (your script).

Linux/Pop!_OS Users Reminder

If the turtle window does not appear in the next step, remember the fix from Lab 1!

- **Host terminal:** Ensure `xhost +local:root` was run.
- **VS Code terminal:** You may need to run `export DISPLAY=:1` (or `:0`) in **EVERY** new terminal tab you open.

1. Open **Terminal 1** and run the simulator:

```
# Linux Users: export DISPLAY=:1
ros2 run turtlesim turtlesim_node

```

2. Open **Terminal 2** (split terminal) and run your new brain:

```
# Don't forget to source!
source install/setup.bash
ros2 run lab2_turtlesim_chase draw_square

```

You should see the turtle drive forward, turn, drive, turn, tracing a rough square path. Ctrl+C to stop the controller; thanks to the try/finally cleanup, the turtle will stop immediately instead of coasting.

Wait, Where is the Subscriber?

In the architecture found in Figure 2, you saw a Subscriber. But if you look closely at the `draw_square.py` code, we never actually create one!
So who is listening to our commands?
The answer is the **simulator itself**. The `turtlesim_node` that you run in the background has a built-in Subscriber listening to the `/turtle1/cmd_vel` topic. Your Python script is strictly a Publisher. It is shouting instructions into the network, and the simulator happens to be listening. Because your node cannot "hear" the turtle's actual position, it is operating entirely **open-loop**. We will fix this by giving your node its own ears (a Subscriber) in Lab 3.

Part 5: Final Task: The Star Navigator

To complete this lab, you must transition from drawing simple squares to drawing a **5-pointed star**. You will modify `draw_square.py` in place.

1. Star Geometry

A 5-pointed star is drawn by moving forward and then turning sharply. The turning angle is **144°** at each corner.

Why 144°? Here's the delightful insight: when you draw a 5-pointed star by going point-to-point, you actually rotate through $5 \times 144 = 720$ (two full rotations) by the time you get back to the start. That extra 360 of rotation is what makes the path cross over itself into a star shape instead of closing into a pentagon. Drawing a regular pentagon would be 72° per corner (totalling 360°); drawing a star doubles it.

Converting to radians: ROS 2 uses radians. $144 \times (\pi/180) \approx$ **2.51 radians**.

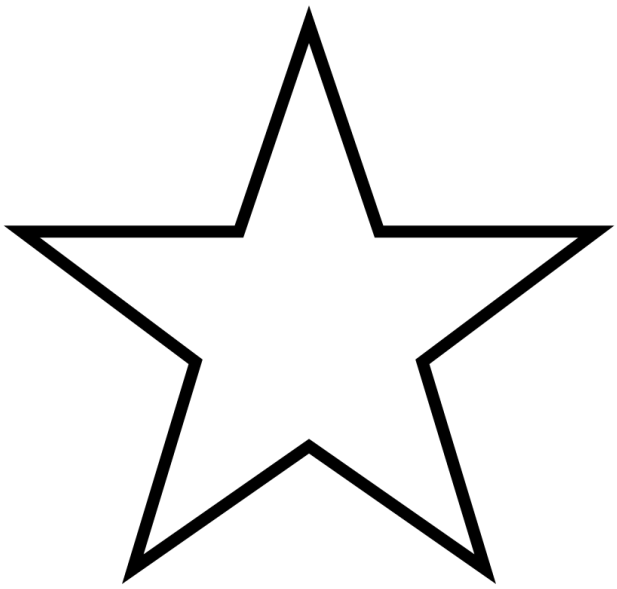

Figure 3: The 5-pointed star path.

2. Modifying Your Logic

Edit `draw_square.py` with one change:

- **The turn:** Change `msg.angular.z` from `1.57` to `2.51`.

That's it. The code loops forever alternating drive/turn, so there's nothing to change about step counts, you just let it run and the star traces out.

Save, then test: Thanks to `--symlink-install`, you do not need to rebuild. Just re-run:

```
ros2 run lab2_turtlesim_chase draw_square
```

Watch the star emerge. When you've got a clean 5-pointed star, you're done.

Want More? (Optional)

Some tuning you might try if you finish early:

- The drive-forward duration is controlled by the `1.0` in `create_timer(1.0, ...)`. Change it to `0.5` and see what happens.
- Change `msg.linear.x = 2.0` to `0.5` for a slower, smaller star.
- Add a step counter that stops the turtle after drawing ONE complete star (10 steps: 5 drives + 5 turns).

These are not required for submission, and they're for students who want to play more.

Troubleshooting

If your code isn't working, check these common issues first:

- **Why does my square drift/shift?**
  Answer: Timer jitter. You are using "open-loop control." You told the robot to turn for 1.0 seconds. If your computer is busy (or the OS lags), the timer might fire at 1.05 seconds.
  - $1.57\ \text{rad/s} \times 0.05\,\text{s} \approx 4.5$ of error per turn!

  This accumulates quickly. To fix this, real robots use sensors (closed-loop control) to verify how far they actually turned. You'll do this in Lab 3.
- **Error: `Package 'lab2_turtlesim_chase' not found.`**
  Solution: You likely forgot to source your setup file. Run this command in every new terminal:

```
source install/setup.bash

```

- **The turtle just spins in circles!**
  Solution: Check your math in `move_turtle_logic`. On odd steps you should be turning with `linear.x = 0.0`; on even steps you should be driving with `angular.z = 0.0`. If both are non-zero on every step, you'll spiral.
- **I pressed Ctrl+C but the turtle kept sliding.**
  Solution: Your try/finally cleanup is missing or broken. Re-check the `main()` block matches the code given in Part 2.

Deliverables

Submit a screenshot of your full desktop showing:

1. The blue Turtlesim window with a clearly drawn **5-pointed star**.
2. The VS Code terminal showing the node successfully running ("Driving forward...", "Turning..." log messages visible).

What's Next: Lab 3

Your square drifts. Your star drifts. You can see that open-loop control isn't good enough for real robots.
In Lab 3 you'll add **subscribers** so your code can read the turtle's actual position, not just guess where it should be. That closes the loop, and it's the difference between "hoping" and "knowing" where your robot is. Welcome to the next step toward a real autonomous tractor.